

15.3.2 Ensuring resource safety

`Process[IO, O]` can be used for talking to external resources like files and database connections, but we must take care to ensure resource safety—we want all file handles to be closed, database connections released, and so on, even (especially!) if exceptions occur. Let's look at what we need to make this happen. We already have most of the machinery in place. The `Await` constructor's `recv` argument can handle errors, choosing to clean up if necessary. And we're catching exceptions in `flatMap` and other relevant combinators so we can ensure that we gracefully pass them to `recv`. All we have to do is make sure that the `recv` function actually calls the necessary cleanup code.

To make the discussion concrete, suppose we have `lines: Process[IO, String]` representing the lines of some large file. This is a *source* or *producer*, and it implicitly references a resource (a file handle) that we want to ensure is closed regardless of how this producer is consumed.

When should we close this file handle? At the very end of our program? No, ideally we'd close the file once we know we're done reading from `lines`. We're certainly done if we reach the last line of the file—at that point there are no more values to produce and it's safe to close the file. So this gives us our first simple rule to follow: *A producer should free any underlying resources as soon as it knows it has no further values to produce, whether due to normal exhaustion or an exception.*

This isn't sufficient though, because the *consumer* of a process may itself decide to terminate consumption early. Consider `runLog { lines("names.txt") |> take(5) }`. The `take(5)` process will halt early after only five elements have been received, possibly before the file has been exhausted. In this case, we want to make sure before halting that any necessary resource freeing is run before the overall process completes. Note that `runLog` can't be responsible for this, since `runLog` has no idea that the `Process` it's interpreting is internally composed of two other `Process` values, one of which requires finalization.

Thus, we have our second simple rule to follow: *Any process d that consumes values from another process p must ensure that cleanup actions of p are run before d halts.*

This sounds rather error prone, but luckily we get to deal with this concern in just a single place, the `|>` combinator. We'll show how that works in section 15.3.3, when we show how to encode single-input processes using our general `Process` type.

So to summarize, a process `p` may terminate due to the following:

- Producer exhaustion, signaled by `End`, when the underlying source has no further values to emit
- Forcible termination, signaled by `Kill`, due to the consumer of `p` indicating it's finished consuming, possibly before the producer `p` is exhausted
- Abnormal termination due to some `e: Throwable` in either the producer or consumer

And no matter the cause, we want to close the underlying resource(s) in each case.

Now that we have our guidelines, how do we actually implement this? We need to make sure the `recv` function in the `Await` constructor always runs the “current” set of cleanup actions when it receives a `Left`. Let’s introduce a new combinator, `onComplete`, which lets us append logic to a `Process` that will be run regardless of how the first `Process` terminates. The definition is similar to `++`, with one twist:

```
def onComplete(p: => Process[F,O]): Process[F,O] =
  this.onHalt {
    case End => p.asFinalizer
    case err => p.asFinalizer ++ Halt(err)
  }
```

Like ++, but *always* runs p, even if this halts with an error.

Helper function

The `p` process is always run when `this` halts, but we take care to re-raise any errors that occur (rather than swallowing them) after running the cleanup action. The `asFinalizer` method converts a “normal” `Process` to one that will invoke itself when given `Kill`. The definition is subtle, but we use this to ensure that in `p1.onComplete(p2)`, `p2` is always run, even if the consumer of the stream wishes to terminate early:

```
def asFinalizer: Process[F,O] = this match {
  case Emit(h, t) => Emit(h, t.asFinalizer)
  case Halt(e) => Halt(e)
  case Await(req,recv) => await(req) {
    case Left(Kill) => this.asFinalizer
    case x => recv(x)
  }
}
```

Putting all this together, we can use the `onComplete` combinator to create a resource-safe `Process[IO,O]` backed by the lines of a file. We define it in terms of the more general combinator, `resource`:

```
def resource[R,O](acquire: IO[R]) (
  use: R => Process[IO,O]) (
  release: R => Process[IO,O]): Process[IO,O] =
  await[IO,R,O](acquire) (r => use(r).onComplete(release(r)))
```



EXERCISE 15.11

This idiom of using `await` to “evaluate” the result of some `IO` action isn’t specific to `IO`. Implement the generic combinator `eval` to promote some `F[A]` to a `Process` that emits only the result of that `F[A]`. Also implement `eval_`, which promotes an `F[A]` to a `Process`, emitting no values. Note that implementing these functions doesn’t require knowing anything about `F`.

```
def eval[F[_],A](a: F[A]): Process[F,A]
def eval_[F[_],A,B](a: F[A]): Process[F,B]
```